

Parser Generators

Concepts of Programming Languages

Announcements

- Assignment 5 due tonight at 8pm
- No assignment to be released today; study
- No lecture next Tuesday (Monday's schedule)
- Midterm 1 next Thursday during lecture
- Students in A2 section (2pm – 3:15pm) can give exam in A1 section (3:30pm – 4:45pm); see Piazza

Example

```
<expr> ::= x  
         | if <expr> then <expr>  
         | if <expr> then <expr> else <expr>
```

What can we do about
ambiguity?

Fixity

The fixity of an operator refers to where an operator is placed relative to its arguments:

prefix

`f x , (- x)`

postfix

`a!`

infix

`a * b , a + b , a mod b`

mixfix

`if b then x else y`

Not Great Solution: Polish Notation

$- \ / \ + \ 2 \ * \ 1 \ - \ 2 \ 3$

is equivalent to

$- \ (2 \ + \ (1 \ * \ (- \ 2) \ / \ 3))$



To avoid ambiguity, we can make **all** operators **prefix** (or postfix) operators. *We don't even need parentheses*

(This how early calculators worked)

Not Great Solution: Lots of Parentheses

<expr>	::=	(<op1> <expr>) (<expr> <op2> <expr>) <var>	((not x) and (not (not y)) or z)
<op1>	::=	not	((((x or y) or z) or x) or y)
<op2>	::=	and or	(not ((not x) and (not y)) or (x and z))
<var>	::=	x y z	(x and y)

If we want infix operators, we *could* add parentheses around all operators

But we run into a similar issue: *Too many parentheses are difficult to read*

Can we get away without,
or with fewer parentheses?

Two Properties of Infix operators

Associativity: *How should arguments be grouped in an expression like $1 + 2 + 3 + 4$?*

Precedence: *How should arguments be grouped in an expression like $1 + 2 * 3 + 4$?*

Associativity

The **associativity** of an infix operator refers to how its arguments are grouped in the absence of parentheses:

left associative

$$1 + 2 + 3 \Rightarrow (1 + 2) + 3$$

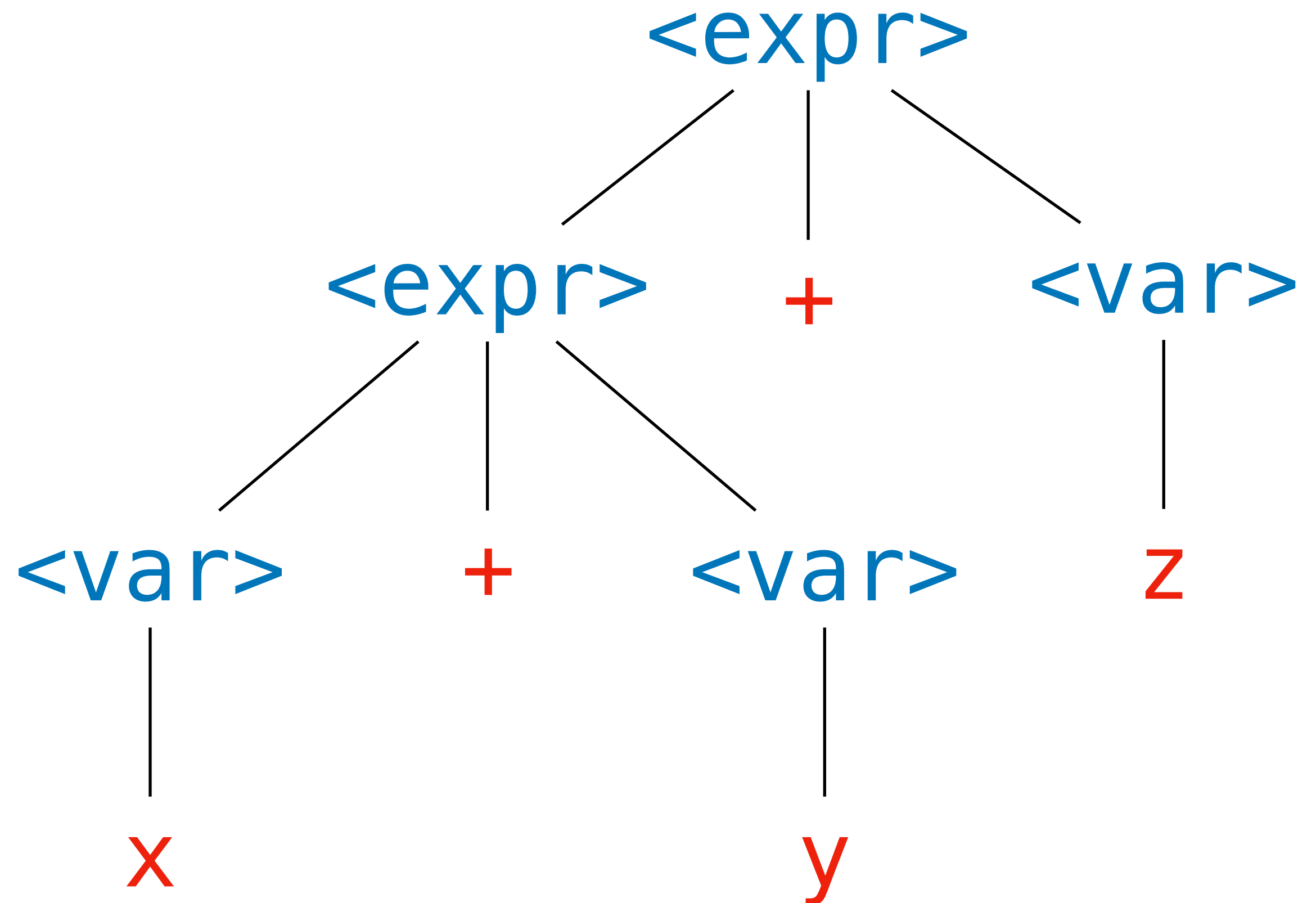
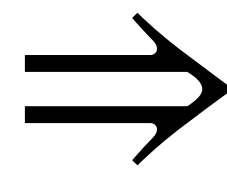
right associative

$$a \rightarrow b \rightarrow c \Rightarrow a \rightarrow (b \rightarrow c)$$

Associativity

<code><expr></code>	<code>::=</code>	<code><expr></code>	<code>+</code>	<code><expr></code>		
				<code><var></code>		
<code><var></code>	<code>::=</code>	<code>x</code>		<code>y</code>		<code>z</code>

`x + y + z`



"add the sum of x and y to z"

How do we enforce that we get a tree of this shape?

The Culprit

$\langle \text{expr} \rangle$	$::=$	$\langle \text{expr} \rangle + \langle \text{expr} \rangle$
	$ $	$\langle \text{var} \rangle$
$\langle \text{var} \rangle$	$::=$	$x \mid y \mid z$

Any time we have a rule like this, we should be suspicious...

$\langle \text{expr} \rangle + \langle \text{expr} \rangle \Rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle + \langle \text{expr} \rangle$

Which $\langle \text{expr} \rangle$ did we replace?

One Solution: Breaking Symmetry

<code><expr></code>	<code>::=</code>	<code><expr></code>	<code>+</code>	<code><var></code>		
		<code> </code>		<code><var></code>		
<code><var></code>	<code>::=</code>	<code>x</code>	<code> </code>	<code>y</code>	<code> </code>	<code>z</code>

```
<expr>
<expr> + <var>
<expr> + z
<expr> + <var> + z
<expr> + y + z
<var> + y + z
x + y + z
```

We make sure that one of the arguments must be "simpler"

By enforcing that the second argument is a `<var>`, we'll get the left-associative parse tree

And For Right Associativity

$\langle \text{type} \rangle$	$::=$	$\langle \text{base} \rangle \rightarrow \langle \text{type} \rangle$
		$\mid \langle \text{base} \rangle$
$\langle \text{base} \rangle$	$::=$	$()$

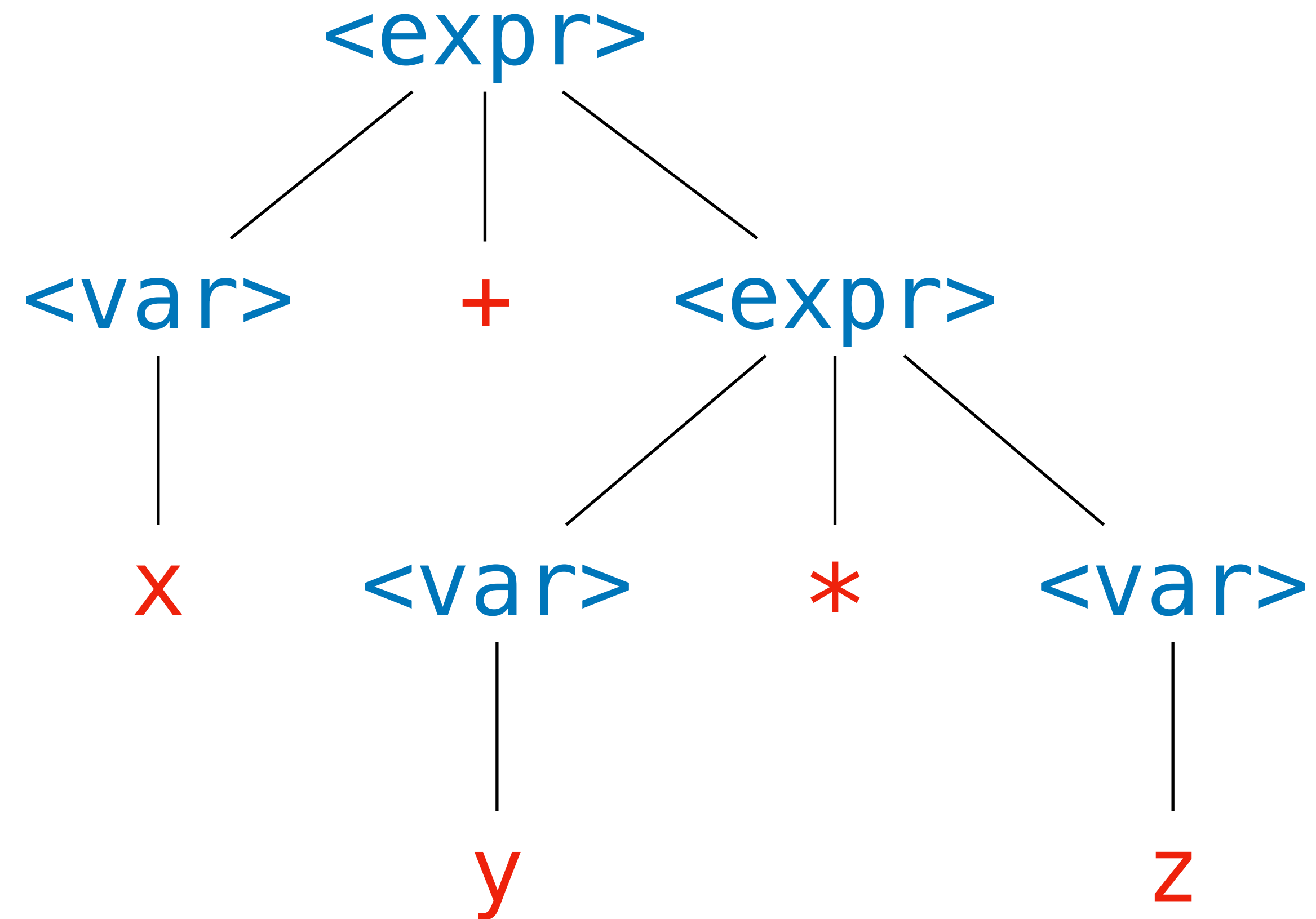
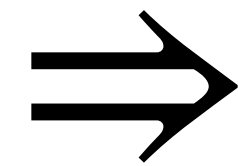
$\langle \text{type} \rangle$
 $\langle \text{base} \rangle \rightarrow \langle \text{type} \rangle$
 $() \rightarrow \langle \text{type} \rangle$
 $() \rightarrow \langle \text{base} \rangle \rightarrow \langle \text{type} \rangle$
 $() \rightarrow () \rightarrow \langle \text{type} \rangle$
 $() \rightarrow () \rightarrow \langle \text{base} \rangle$
 $() \rightarrow () \rightarrow ()$

For right associativity, we break symmetry by "factoring out" the *left* argument

Multiple Operators

<code><expr></code>	<code>::=</code>	<code><expr></code>	<code><op></code>	<code><var></code>
		<code><var></code>		
<code><op></code>	<code>::=</code>	<code>+</code>	<code>*</code>	
<code><var></code>	<code>::=</code>	<code>x</code>	<code>y</code>	<code>z</code>

`x + y * z`



"add x to the product of y and z"

Question. What if we have multiple operators? Which one should "bind tighter"?

Precedence

$$2 + 3 \times 6 = 2 + (3 \times 6) = 20$$

The precedence of an operator refers to order in which an operator should be considered, relative to other operators

Example. **PEMDAS** (paren, exp, mul, div, add, sub)

Higher precedence means it "binds tighter"

Dealing with Precedence within the Grammar

<code><expr></code>	<code>::=</code>	<code><expr></code>	<code>+</code>	<code><term></code>
				<code><term></code>
<code><term></code>	<code>::=</code>	<code><term></code>	<code>*</code>	<code><var></code>
				<code><var></code>
<code><var></code>	<code>::=</code>	<code>x</code>		<code>y</code> <code>z</code>

We factor out the `*` part of the `<expr>` rule

Note that we handle *lower* precedence terms first, since terms *deeper* in the parse tree are evaluated first

Example

<expr>

<term>

<pars>

<var>

::=

::=

::=

::=

<expr>

<term>

<var>

x

y

z

+

*

x + y * z

Example

<expr>

<term>

<pars>

<var>

::=

::=

::=

::=

<expr>

<term>

<var>

x

y

z

+

*

<term>

<pars>

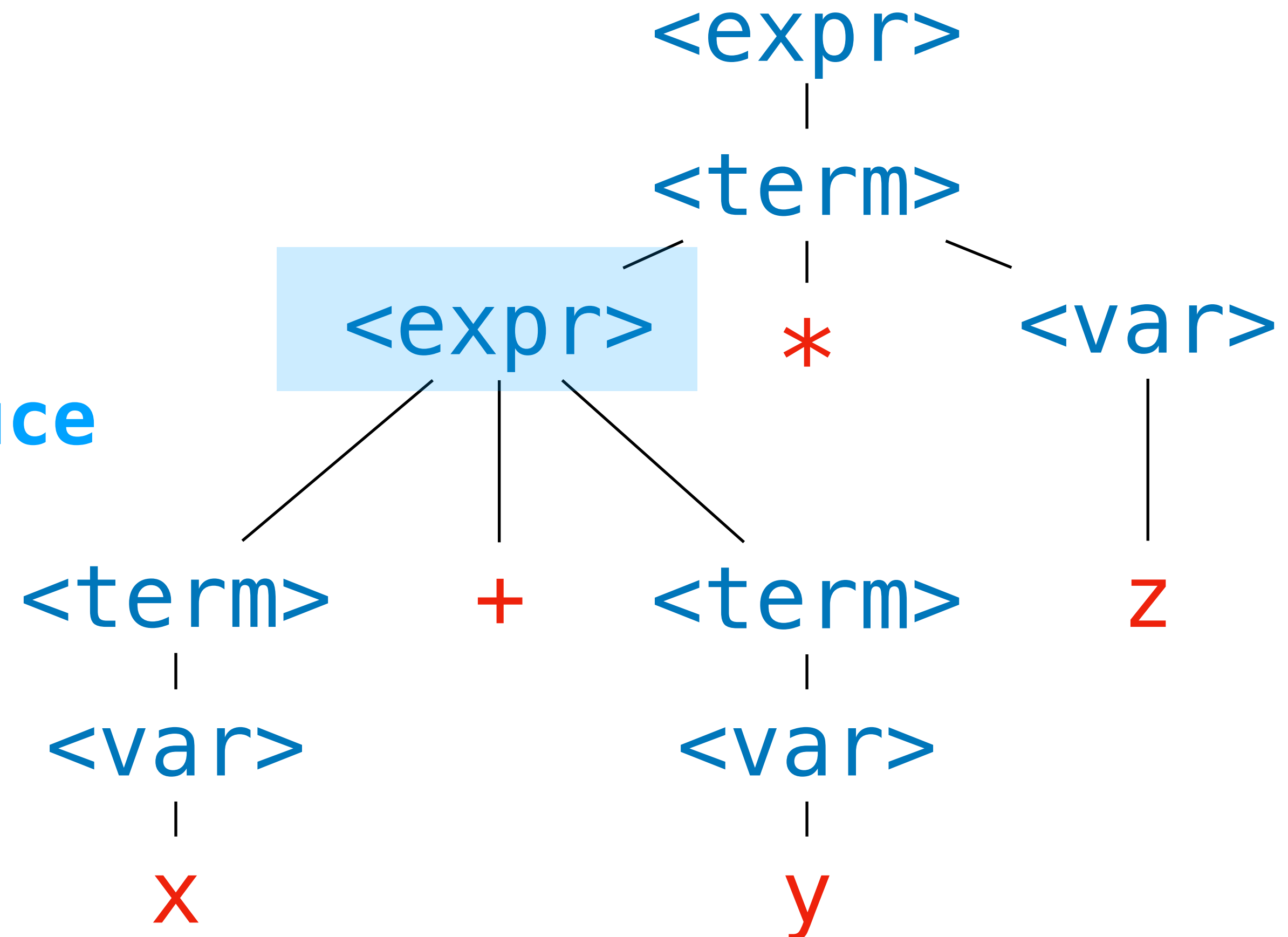
x * y + z

The Issue of Parentheses Returns

<code><expr></code>	<code>::=</code>	<code><expr></code>	<code>+</code>	<code><term></code>		
				<code><term></code>		
<code><term></code>	<code>::=</code>	<code><term></code>	<code>*</code>	<code><var></code>		
				<code><var></code>		
<code><var></code>	<code>::=</code>	<code>x</code>		<code>y</code>		<code>z</code>

No, we need to introduce parentheses again

`( x + y ) * z`



"multiply the sum of x and y with z"

Question. Can we derive this parse tree?

Dealing with Parentheses

<code><expr></code>	<code>::=</code>	<code><expr></code>	<code>+</code>	<code><term></code>
		<code> </code>		<code><term></code>
<code><term></code>	<code>::=</code>	<code><term></code>	<code>*</code>	<code><var></code>
		<code> </code>		<code><pars></code>
<code><pars></code>	<code>::=</code>	<code><var></code>	<code> </code>	<code>(<expr>)</code>
<code><var></code>	<code>::=</code>	<code>x</code>	<code> </code>	<code>y</code> <code> </code> <code>z</code>

We further factor out the part of the rule for parentheses. Note that **any expression** can appear in the parentheses

(This is a circular, or mutually recursive, production rule)

Outline

- » Extend our BNF syntax to be a bit more convenient
- » Introduce **parser generators**
- » Demo **Menhir**, the parser generator for this course

Extended BNF

Extended BNF

<code><expr></code>	<code>::=</code>	<code><only-mul-div></code>	<code>{</code>	<code>(+ -)</code>	<code><only-mul-div></code>	<code>}</code>
<code><only-mul-div></code>	<code>::=</code>	<code><var-or-parens></code>	<code>{</code>	<code>(* /)</code>	<code><var-or-parens></code>	<code>}</code>
<code><var-or-parens></code>	<code>::=</code>	<code>x (<expr>)</code>				

Extended BNF is syntactic sugar. **It is no more expressive than BNF**

But it allows us to specify:

- » Optional parts of production rule
- » Repeated parts of a production rule

Note: EBNF means different things to different people

Optional Syntax

BNF : `<expr> ::= if <expr> then <expr>
 | if <expr> then <expr> else <expr>`

EBNF : `<expr> ::= if <expr> then <expr> [ else <expr> ]`

Menhir :

```
expr:
  | IF; e1 = expr; THEN; e2 = expr; e3_opt = else?
    { match e3_opt with
      | None -> It (e1, e2)
      | Some e3 -> Ite (e1, e2, e3)
    }
else:
  | ELSE; e = expr { e }
```

Repetition Syntax

BNF :

```
<expr> ::= [ ] | [ <elems> ]  
<elems> ::= <expr> | <expr> ; <elems>
```

EBNF :

```
<expr> ::= [ ] | [ <expr> { <elem> } ]  
<elem> ::= ; <expr>
```

Menhir :

```
elem:  
  | SEMICOLON; e = expr { e }  
expr:  
  | LBRAK; RBRAK { List_lit [] }  
  | LBRAK; e = expr; es = elem*; RBRAK  
    { List_lit (e :: es) }
```

Interlude: Regular Expressions

Regular Expressions (Formally)

Regular expressions (Regex) provide a compact way of describing regular grammars:

- » A **terminal symbols** is a regex
- » **[t1 ... tk]** is a regex describing an **any one of** the symbols **t1, t2, ..., tk**
- » **(e1 | ... | ek)** is a regex describing any one of the expressions **e1, e2, ..., ek**
- » **exp*** is a regex describing **zero or more occurrences** of **exp**
- » **exp+** is a regex describing **one or more occurrences** of **exp**
- » **exp?** is a regex describing **zero or one occurrences** of **exp**

Example

$\langle s \rangle ::= a \langle s \rangle$
 $\langle s \rangle ::= b \langle a \rangle$
 $\langle a \rangle ::= \epsilon$
 $\langle a \rangle ::= c \langle a \rangle$

is equivalent to

$a*bc*$

or

$'a'* 'b' 'c'*$

in ocamllex syntax

Example: Numbers and Variables

`-?[0-9]+`

numbers

`[a-z_][a-z0-9A-Z_']*`

variables

We'll leave it there, take CS332
if you want more, or read the
Wikipedia page...

Parsing with Menhir

General Parsing

In Theory. Determine if a given sentence is recognized by a given grammar

In Practice. Given a grammar, write a program which converts a string recognized by that grammar into an ADT

Today

```
<prog> ::= <expr>

<expr> ::= let <var> = <expr> in <expr>
        | <expr1>

<bop>   ::= + | - | * | /

<expr1> ::= <expr1> <bop> <expr1>
        | <num>
        | <var>
        | ( <expr> )

<num>   ::= 0 ; DUMMY VALUE
<var>   ::= x ; DUMMY VALUE

; In lex.mll:
;
; let num = '-'? ['0'-'9']+
; let var = ['a'-'z' '_' ] ['a'-'z' 'A'-'Z' '0'-'9' '_' '\''']*

Operators in order of increasing precedence:
```

Operator	Associativity
+, -	left
*, /	left

We'll be building a parser for this grammar

A Note on Associativity and Precedence

```
%token PLUS  
%token MINUS  
%token TIMES  
%token DIVIDE
```

```
%left PLUS, MINUS  
%left TIMES, DIVIDE
```

In Menhir, we actually won't deal with associativity and precedence using new non-terminals

Using a parser generator we'll often be able to *specify* the precedence of an operator

A Rough Sketch

1. Specify the tokens (i.e., terminal symbols) of the grammar
2. Specify the rules of the grammar (using a BNF-like syntax)
3. Specify the rules of the lexer (i.e., which strings go to which tokens)